

CSC 220 - Java Review Note

Core Java & Object-Oriented Programming Quick Reference

Use this as a review/reference sheet while coding. It is not a replacement for practice.

1. JDK, JRE, and JVM

Java source code is compiled into platform-independent bytecode, which is executed by the Java Virtual Machine (JVM).

Java Source (.java) --javac--> Bytecode (.class) --JVM--> Running Program

- JVM (Java Virtual Machine): executes Java bytecode.
- JRE (Java Runtime Environment): provides the runtime environment, including the JVM and Java libraries.
- JDK (Java Development Kit): provides development tools such as javac, java, and javadoc, together with the runtime components.

Mental model: JDK -> runtime environment -> JVM.

2. Anatomy of a Java Class

```
package model.vehicle;
```

```
public class Car {  
    private String vin;  
    private String make;  
    private String model;  
  
    public Car(String vin, String make, String model) {  
        this.vin = vin;  
        this.make = make;  
        this.model = model;  
    }  
  
    public String getVin() {  
        return vin;  
    }  
  
    public void start() {  
        System.out.println("Car started.");  
    }  
}
```

State -> fields/instance variables | Behavior -> methods | Creation -> constructors

```
Car myCar = new Car("VIN123", "Honda", "CR-V");
```

3. Constructors and Overloading

A constructor initializes a new object. It has the same name as the class and has no return type.

```
public Car(String vin) {  
    this.vin = vin;  
}  
  
public Car(String vin, String make, String model) {
```

```

    this.vin = vin;
    this.make = make;
    this.model = model;
}

// Copy-style overloaded constructor
public Car(Car other) {
    this.vin = other.vin;
    this.make = other.make;
    this.model = other.model;
}

```

Same name + different parameter list = overloading.

4. Access Modifiers

Modifier	Same Class	Same Package	Subclass*	Everywhere
private	Yes	No	No	No
default	Yes	Yes	No	No
protected	Yes	Yes	Yes	No
public	Yes	Yes	Yes	Yes

*Protected access across packages has additional Java rules. Use private fields unless broader access is actually needed.

5. Encapsulation

Encapsulation protects an object's internal state and provides controlled ways to access or modify it.

```

private double perHourRate;

public double getPerHourRate() {
    return perHourRate;
}

public void setPerHourRate(double rate) {
    if (rate >= 0) {
        perHourRate = rate;
    }
}

```

Encapsulation is more than adding getters/setters: hide internal state and control valid changes.

6. Packages

```

src/
|-- app/
|   |-- App.java
|-- model/
|   |-- vehicle/
|       |-- Vehicle.java
|       |-- Car.java
|       |-- Motorcycle.java
// Car.java
package model.vehicle;

// App.java
import model.vehicle.Car;

```

The package name should correspond to the directory structure relative to the source root.

7. Inheritance - IS-A

```
public abstract class Vehicle {
    private String vin;

    public Vehicle(String vin) {
        this.vin = vin;
    }

    public String getVin() {
        return vin;
    }
}

public class Car extends Vehicle {
    public Car(String vin) {
        super(vin);
    }
}
```

Car IS-A Vehicle. `super(...)` invokes a superclass constructor.

8. Every Java Class Inherits from Object

```
Object
 ^
Vehicle
 ^
Car
```

`java.lang.Object` is at the top of the Java class hierarchy. Important inherited methods include `toString()`, `equals(Object)`, `hashCode()`, and `getClass()`.

9. `toString()`

```
@Override
public String toString() {
    return vin + ": " + make + " " + model;
}
```

Overriding `toString()` gives an object a useful textual representation when it is printed.

10. Overloading vs. Overriding

Feature	Overloading	Overriding
Method name	Same	Same
Parameters	Different	Same signature
Inheritance required?	No	Yes
Resolution	Compile time	Runtime dispatch
<code>@Override</code>	No	Yes

```
// Overloading
public void park() {}
public void park(int hours) {}
```

```
// Overriding
```

```

@Override
public void start() {
    System.out.println("Car started.");
}

```

11. Be Careful with equals()

Object defines: public boolean equals(Object obj)

```

public boolean equals(Car other) {
    // This OVERLOADS equals; it does not override Object.equals().
}

```

```

@Override
public boolean equals(Object obj) {
    // This can OVERRIDE Object.equals().
}

```

Changing the parameter from Object to Car changes the method signature. In production Java, a class that overrides equals() should also provide a compatible hashCode().

12. Abstract Classes

```

public abstract class Vehicle {
    public abstract void start();
    public abstract void stop();
}

public class Car extends Vehicle {
    @Override
    public void start() {
        System.out.println("Car started.");
    }

    @Override
    public void stop() {
        System.out.println("Car stopped.");
    }
}

```

An abstract class can contain fields, constructors, concrete methods, and abstract methods. It cannot be instantiated directly.

13. Polymorphism

Polymorphism allows objects of different subclasses to be treated through a common type while providing their own implementations of behavior.

```

Vehicle[] vehicles = {
    new Car(...),
    new Motorcycle(...),
    new ElectricCar(...)
};

for (Vehicle vehicle : vehicles) {

```

```

    vehicle.start();
}

```

One common type, many forms of behavior.

14. Interface vs. Abstract Class

```

public interface Chargeable {
    void charge();
}

```

```

public class ElectricCar extends Vehicle implements Chargeable {
    @Override
    public void charge() {
        // charging behavior
    }
}

```

- Abstract class: useful for common state and shared/inherited behavior.
- Interface: useful for defining a contract or capability.
- A class can extend one class but implement multiple interfaces.

Useful guideline: extends -> What am I? implements -> What can I do?

15. Quick Syntax Reference

```

Car c = new Car(...);           // create object
class Car extends Vehicle { }   // inheritance
class EV implements Chargeable { } // interface
super(vin);                     // superclass constructor
this.vin = vin;                 // current object
@Override                       // compiler-checked override
obj instanceof Car              // type check
Car c2 = (Car) obj;             // cast

```

Quick Memory Check

- Class -> blueprint for objects
- Object -> instance of a class
- Field -> object's state/data
- Method -> object's behavior
- Constructor -> initializes an object
- Inheritance -> IS-A relationship
- Encapsulation -> protect/control object state
- Overloading -> same name, different parameters
- Overriding -> subtype supplies inherited method behavior
- Polymorphism -> common type, different runtime behavior
- Interface -> behavioral contract/capability
- Package -> organizes related Java types